Module 5 — Software Assurance: Hardened Flask + PostgreSQL App

Name: Vishal Srivastava Course: JHU Software GitHub: https://github.com/vsrivas7/jhu_software_concepts

1. How to Install and Run

Option A — pip + venv:

```
cd module_5
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install -e .
cp .env.example .env
flask --app src.flask_app:create_app run
```

Option B — uv:

```
cd module_5
uv venv .uvenv
uv pip sync requirements.txt --python .uvenv/bin/python
uv pip install -e . --python .uvenv/bin/python
cp .env.example .env
flask --app src.flask_app:create_app run
```

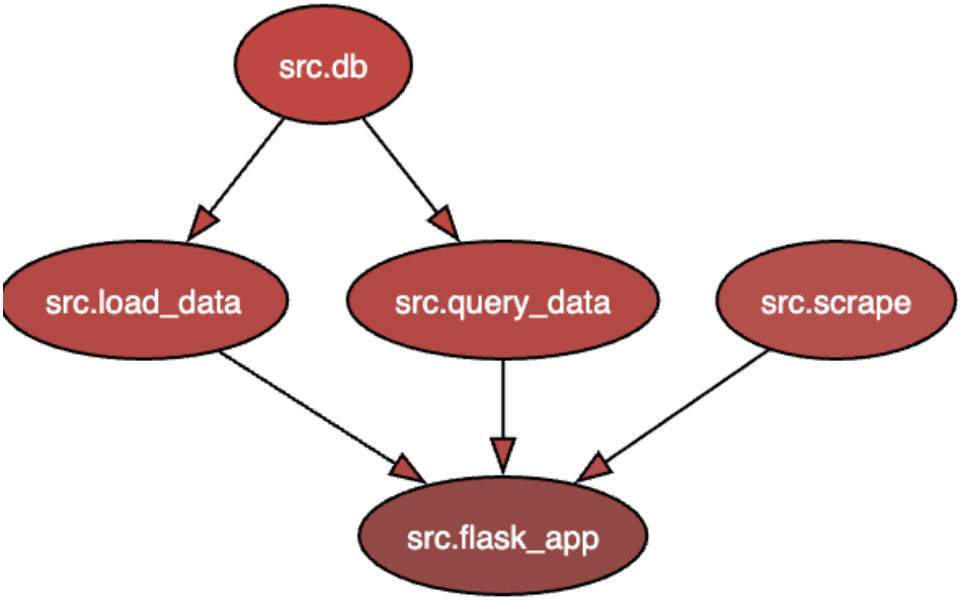
Screenshot of pylint 10/10

```
zsh: command not found: your
(venv) (base) vishal@Vishals-MBP module_5 % cd /Users/vishal/jhu_software_concepts/module_5
source venv/bin/activate
pylint src/ --fail-under=10

------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

(venv) (base) vishal@Vishals-MBP module_5 %
```

2. Python Dependency Graph

The graph shows that src.flask_app is the central entry point of the application, depending on all three sibling modules: src.load_data, src.query_data, and src.scrape. src.db sits at the top of the hierarchy as a shared utility, providing the get_connection() function to both src.load_data and src.query_data. This design eliminates code duplication and centralises all database credential logic in one place. src.scrape is an independent module with no database dependency — it only handles HTTP requests and HTML parsing via requests and beautifulsoup4. src.load_data

handles INSERT operations into PostgreSQL, relying on src.db for its connection. src.query_data handles all SELECT/analysis queries, also using src.db for its connection. The architecture is clean with no circular dependencies and a clear separation of concerns across scraping, loading, querying, and routing layers.



3. SQL Injection Defenses

The original code built SQL queries using string concatenation and f-strings, which allowed user input to be directly embedded into SQL text — a classic SQL injection vulnerability. In module_5, all SQL was rewritten using psycopg's safe SQL composition library.

What we use now and why it's safe:

Technique	Used for	Example
sql.SQL(...)	Statement structure	sql.SQL("SELECT ... FROM {table} LIMIT %s")
sql.Identifier(...)	Table/column names	.format(table=sql.Identifier("applicants"))
%s + params tuple	All user-supplied values	cur.execute(stmt, ("%fall%", safe_limit))

Key requirements met:

- No f-strings or string concatenation in SQL construction
- Dynamic identifiers (table/column names) safely quoted via sql.Identifier
- User values never appear in SQL text — passed as separate params to cursor.execute()
- SQL statement construction is fully separated from execution

- Every query has an explicit LIMIT %s with _clamp_limit() clamping values between 1 and 100, preventing unbounded data dumps

4. Least-Privilege Database Configuration

The application only needs to read from and insert into the applicants table. No feature requires UPDATE, DELETE, DROP, or ALTER, so those are intentionally withheld.

SQL used to create the least-privilege user:

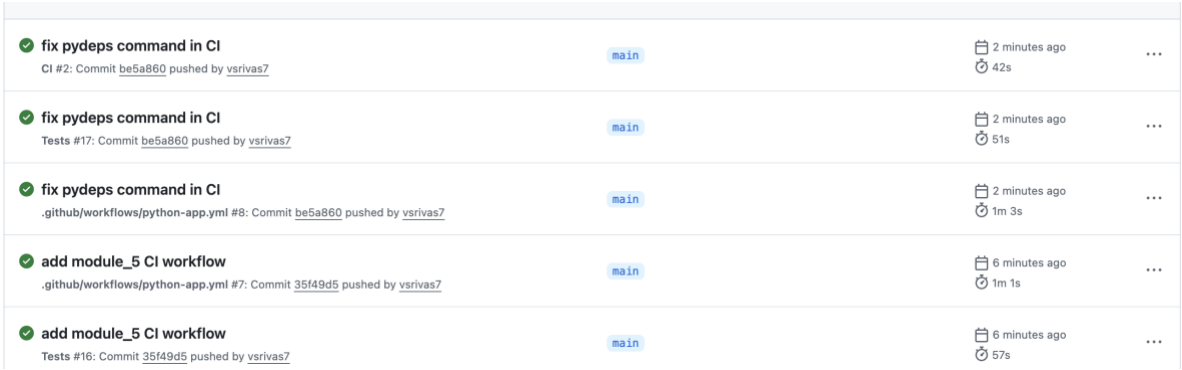
sql
```
-- Create a restricted user (not a superuser)
CREATE USER gradcafe_user WITH PASSWORD 'yourpassword';

-- Grant connection access to the database
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_user;

-- Grant usage on the schema
GRANT USAGE ON SCHEMA public TO gradcafe_user;

-- Grant only SELECT and INSERT on the specific table
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_user;
```

- SELECT — needed by /analysis endpoint to read applicant data
- INSERT — needed by /pull-data endpoint to store scraped rows
- No UPDATE, DELETE, DROP, or ALTER — not required by any feature
- Not a superuser — confirmed by empty Attributes column in \du output
- If the account is compromised, an attacker cannot drop tables or modify schema

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS                                   psql

(venv) (base) vishal@Vishals-MBP module_5 % cat .env.example
cat .gitignore | grep .env
.venv/
(venv) (base) vishal@Vishals-MBP module_5 % psql --version
psql (PostgreSQL) 14.20 (Homebrew)
(venv) (base) vishal@Vishals-MBP module_5 % psql -U postgres -d gradcafe
psql: error: connection to server on socket "/tmp/.s.PGSQL.5432" failed: FATAL:  role "postgres" does not exist
(venv) (base) vishal@Vishals-MBP module_5 % psql -d gradcafe
psql (14.20 (Homebrew))
Type "help" for help.

gradcafe=# CREATE USER gradcafe_user WITH PASSWORD 'yourpassword';
CREATE ROLE
gradcafe=# GRANT CONNECT ON DATABASE gradcafe TO gradcafe_user;
GRANT
gradcafe=# GRANT USAGE ON SCHEMA public TO gradcafe_user;
GRANT
gradcafe=# GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_user;
GRANT
gradcafe=#
gradcafe=# \du gradcafe_user
                List of roles
   Role name   | Attributes | Member of
---------------+------------+-----------
 gradcafe_user |            | {}
```

5. GitHub Actions CI

The workflow at .github/workflows/ci.yml runs on every push and pull request with four separate jobs:



- Pylint — runs pylint src/ --fail-under=10, fails if score is below 10.00/10
- Pytest — runs full test suite with coverage via pytest tests/ -v --cov=src
- Generate dependency.svg — installs Graphviz, runs pydeps src, fails if SVG not generated
- Snyk Security Scan — runs snyk test against requirements.txt using SNYK_TOKEN secret

6. Snyk Dependency Scan Results

- snyk test — Tested 38 dependencies, 0 vulnerabilities found
- snyk code test (Extra Credit) — Static code analysis, 0 issues found

```
Docs:    https://docs.snyk.io/scan-with-snyk/error-catalog#snyk-code-0005

ID:      urn:snyk:interaction:9e409ba9-3321-420d-9c1d-e15bcf736256
(venv) (base) vishal@Vishals-MBP module_5 % snyk code test

Testing /Users/vishal/jhu_software_concepts/module_5 ...


  Test Summary

    Organization:      vsrivas7
    Test type:         Static code analysis
    Project path:      /Users/vishal/jhu_software_concepts/module_5

    Total issues:   0

  Tip

    To view ignored issues, use the --include-ignores option.


(venv) (base) vishal@Vishals-MBP module_5 %
```